



SIMATS SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
CHENNAI-602105



ASSESSMENT 4

CSA1006-SOFTWARE ENGINEERING

Submitted By

Name: Harendra.G

Register Number: 192521215

Department: Information Technology

Year: 2nd, B.Tech – IT

Faculty Name: Dr.Surendiren

PROBLEM 1:

**Explain Mutation Testing and Fuzz Testing.
How do these testing techniques enhance the
reliability and security of software systems?**

Mutation Testing and Fuzz Testing are software testing techniques used to find defects and improve the quality, reliability, and security of software systems.

A. Mutation Testing

Mutation Testing is a testing technique used to check whether existing test cases are effective.

In mutation testing, small changes called **mutants** are intentionally introduced into the program. The test cases are then executed to check whether they can detect these changes.

Example:

Original code:

```
if (age >= 18)
    allow voting;
```

Mutated code:

```
if (age > 18)
    allow voting;
```

A good test case should detect this change and fail. If the test cases still pass, it means the test suite may not be strong enough.

Steps in Mutation Testing

1. Start with the original program.
2. Create small changes or mutations in the code.
3. Run the existing test cases.
4. If a test fails, the mutant is called **killed**.
5. If all tests pass, the mutant **survives**.
6. Improve the test cases for surviving mutants.

B. Fuzz Testing

- **Fuzz Testing**, or **Fuzzing**, is a testing technique in which a large number of unexpected, random, or invalid inputs are given to a software application.
- The purpose is to find crashes, errors, security vulnerabilities, and unexpected behavior.

Example:

For a login system, a fuzz tester may provide:

- . Very long usernames
- . Empty inputs
- . Special characters
- . Random numbers

- Unexpected symbols
- Very large input values

If the application crashes or behaves incorrectly, the problem can be identified and fixed.

How They Improve Reliability and Security

Technique	Main Benefit
Mutation Testing	Checks the effectiveness of test cases
Fuzz Testing	Finds unexpected errors and security vulnerabilities

Mutation Testing improves reliability by ensuring that test cases can detect changes and defects in the program.

Fuzz Testing improves security by finding weaknesses that may occur when unexpected or malicious-looking inputs are provided.

Conclusion

Mutation Testing makes sure that the **tests themselves are strong**, while Fuzz Testing checks how the **software behaves with unexpected inputs**. Together, they help developers create software that is more reliable, robust, and secure.

PROBLEM 2:

Describe the Key Stages of a CI/CD Pipeline (using Jenkins or GitHub Actions). How does using Docker for containerization support continuous delivery?

A **CI/CD pipeline** is an automated process used to build, test, and deliver software. Jenkins and GitHub Actions are commonly used to implement CI/CD pipelines. The assignment specifically asks about the stages of a CI/CD pipeline and the role of Docker in continuous delivery.

Key Stages of a CI/CD Pipeline

1. Source Code Management

- Developers write and modify the application code and store it in a version-control system such as GitHub.
- When new code is pushed to the repository, the CI/CD pipeline can automatically start.

2. Build

The pipeline automatically builds the application.

For example, it can:

- Compile source code
- Install dependencies
- Create executable files or packages
- Check for basic build errors

If the build fails, the pipeline stops and the developer can fix the problem.

3. Testing

Automated tests are executed to check whether the application works correctly.

Testing can include:

- Unit testing
- Integration testing
- Functional testing
- Security testing

If tests fail, the software is not moved to the next stage.

4. Package

After successful testing, the application is packaged for deployment.

For example, the application can be packaged into a **Docker image**.

5. Deployment

The application is automatically deployed to a target environment such as:

- Development
- Testing
- Staging
- Production

6. Monitoring and Feedback

- After deployment, the application is monitored to identify errors and performance problems.
- Feedback can be used by developers to make improvements and release the next version.

Simple CI/CD Flow

Developer



GitHub Repository



Build



Automated Testing



Docker Image



Deployment



Monitoring



Feedback

How Docker Supports Continuous Delivery

Docker is a containerization technology that packages an application along with its required libraries, dependencies, and configuration into a container.

For example:

Application

+ Dependencies

+ Libraries

+ Configuration



Docker Image



Container

Benefits of Docker in CI/CD

1. Consistent Environment

Docker ensures that the application runs in a similar environment during development, testing, and production.

2. Easy Deployment

A Docker image created during the pipeline can be deployed directly to the target environment.

3. Faster Delivery

Containers can be created and started quickly, helping applications move through the delivery pipeline faster.

4. Dependency Management

Application dependencies are included in the container, reducing problems caused by different versions of libraries or software.

5. Portability

Docker containers can run on different systems that support Docker.

6. Easy Rollback

Previous Docker images can be retained. If a new version has a problem, the previous version can be deployed again.

Conclusion

A CI/CD pipeline automates the process from **code commit to deployment**. Docker supports continuous delivery by providing a consistent, portable, and repeatable environment for building, testing, and deploying applications.

PROBLEM 3:

Case Study – Intermittent Slowdowns in a Large SaaS Application

A large **SaaS (Software as a Service)** application may sometimes become slow because of high CPU usage, memory problems, database delays, network issues, or overloaded services. Monitoring and logging tools such as **Prometheus, Grafana, and the ELK Stack** can help detect, diagnose, and resolve these performance problems.

1. Prometheus – Monitoring and Metrics

Prometheus is a monitoring system that collects and stores numerical metrics from applications and infrastructure.

It can monitor:

- CPU usage
- Memory usage
- Request rate
- Response time
- Error rate

- Disk usage
- Network activity

For example, if the SaaS application suddenly becomes slow, Prometheus can show that CPU usage increased significantly during the same period.

Role of Prometheus

Application/Servers



Metrics



Prometheus



Performance Data

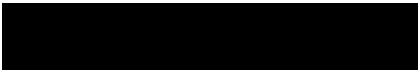
This helps identify when a problem started and which system resources are affected.

2. Grafana – Visualization and Analysis

Grafana is used to display monitoring data through dashboards and graphs.

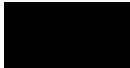
Prometheus can provide the metrics, while Grafana presents them in an easy-to-understand visual form.

For example, a Grafana dashboard can show:

CPU Usage  90%

Memory Usage  80%

Response Time  High

Error Rate  Increasing

This makes it easier for developers and system administrators to identify unusual patterns.

Role of Grafana

Grafana helps answer questions such as:

- When did the slowdown begin?
- Which server is affected?
- Is CPU usage increasing?

- Is response time higher than normal?
 - Are errors increasing?
-

3. ELK Stack – Log Analysis

The **ELK Stack** consists of:

- **Elasticsearch**
- **Logstash**
- **Kibana**

Elasticsearch

Elasticsearch stores and searches large amounts of log data.

Logstash

Logstash collects logs from different sources and processes them.

Kibana

Kibana provides dashboards and visualizations for analyzing the logs.

ELK Flow

Applications / Servers



Logstash



Elasticsearch



Kibana

For example, if users experience slow responses, logs can be searched to find:

- Application errors
- Database errors
- Failed requests
- Timeout messages
- Service failures

4. Detecting the Problem

The tools can work together to detect intermittent slowdowns.

For example:

1. Users report that the application is slow.
2. Prometheus shows an increase in response time.
3. Grafana displays the increase on a dashboard.
4. ELK logs are searched for errors during the same period.
5. Developers identify the affected service or operation.

5. Diagnosing the Problem

The collected metrics and logs help determine the root cause.

For example:

High Response Time



Check Prometheus Metrics



High CPU detected



Check ELK Logs



Find repeated errors



Identify faulty service

This provides both **numerical performance information** and **detailed application logs**.

6. Resolving the Problem

After finding the cause, developers can take appropriate action.

Examples include:

- Optimizing slow code

- Fixing application errors
- Increasing server resources
- Scaling services
- Optimizing database queries
- Fixing network problems
- Restarting or replacing faulty services

After the fix is deployed, Prometheus and Grafana can be used to verify whether performance has improved.

Conclusion

Prometheus, Grafana, and the ELK Stack provide complementary capabilities for managing SaaS performance.

- **Prometheus** → collects performance metrics.
- **Grafana** → displays and analyzes metrics.
- **ELK Stack** → collects, searches, and analyzes application logs.

